

Powtórzenie przed kolokwium nr I

1. Użytkownik podaje jednostkę: "km", "cm" lub "mm" oraz liczbę np. 3035.34. Program ma przekonwertować daną wartość na metry, w zależności od tego, jaka jednostka została podana, i wyświetlić odpowiedni komunikat, np:

2 km to 2000 m

- o Możemy użyć funkcji `strcmp` z biblioteki `<string.h>`, aby prosto porównać dane wejściowe od użytkownika (o jednostce) z naszymi założonymi wariantami (kilometr, centymetr lub milimetr).

```
#include <string.h>

/*
(...)
*/
char str1[2] = "ab";
char str2[2] = "ba";
if (strcmp(str1, str2) == 0) {
    printf("str1 i str2 są równe.");
} else {
    printf("str1 i str2 nie są równe.");
}
/*
(...)
*/
```

2. Użytkownik podaje, ile ma kart oraz ile osób chce zagrać w jakąś grę karcianą, w której każdy ma dostać po tyle samo kart. Sprawdzić, czy da się rozdzielić podaną liczbę kart po równo między graczy. Jeśli nie, to wyświetlić na ekranie, ile osób dostało by o jedną kartę mniej od reszty, jak byśmy i tak rozdawali karty po jednej dla każdego w określonej kolejności.
3. Za pomocą pętli, znaleźć liczbę dzielników liczby naturalnej n , którą poda użytkownik. Wypisać przy okazji na ekranie każdy znaleziony dzielnik liczby n .
4. Wypełnić kwadratową tablicę `tab` różnymi liczbami i wyświetlić na ekranie dolną trójkątną część tablicy `tab`, w której dodatkowo liczby w każdej kolumnie (wymiar n dla np. `tab[m][n]`), są uporządkowane w odwrotnej kolejności do pierwotnej. Można się posłużyć np. dodatkową, pomocniczą tablicą, aby to uzyskać, albo podmienić na początku kolejność elementów w każdej z kolumn, itd.
 - o Kod ma działać dla dowolnego rozmiaru tablicy `N`, ale założmy, że tablica to:

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

- To dolna trójkątna część tej tablicy to:

6			
11	12		
16	17	18	
21	22	23	24

- Jednakże dodatkowo w naszym zadaniu oryginalna tablica ma być odwrócona w kolumnach, więc oczekujemy:

16			
11	12		
6	7	8	
1	2	3	4

5. Zaimplementować poniższy pseudokod algorytmu sortowania InsertSort, jako funkcję `void insertSort(int* A, int length)`. **Dodatkowo:** zwizualizować proces sortowania, dodając instrukcję drukującą na ekranie całą tabelę `A` po każdym przejściu wewnętrznej pętli `WHILE`.

insertSort($A \in \mathbb{N}^{\text{length}}$) :

```

i := 1
WHILE (i < length)

    j := i
    WHILE (j > 0 AND A[j-1] > A[j])

        temp := A[j]
        A[j] := A[j-1]
        A[j-1] := temp

        j := j - 1
    END WHILE

    i := i + 1
END WHILE

```

6. Zaimplementować jako **funkcje rekurencyjne** poniższe zależności rekurencyjne:

1. Ciąg Padovana $P(n)$, dla $n \in \mathbb{N}$:

- 1, gdy $n = 0, 1, 2$
- $P(n - 2) + P(n - 3)$, gdy $n \geq 3$

2. suma cyfr liczby $S(n)$, dla $n \in \mathbb{N}$:

- $S(n) = n$, gdy $n < 10$
- $(n \bmod 10) + S(n//10)$ w każdym innym przypadku

$//$ to dzielenie w liczbach całkowitych, np $5//2 = 2$, $4//2 = 2$, $8//3 = 2$, $2//3 = 0$.
Natomiast $n \bmod m$ to reszta z dzielenia n przez m .

$$S(5273) = 3 + 7 + 2 + 5 = 17.$$